Source Fetchers — typed adapter contract for external knowledge sources

SPEC-TH-011 v0.1 DRAFT Jeremie Nunez

1 Context

Thalamus cortices enrich their reasoning with data from heterogeneous external sources (climate archives, terroir databases, market feeds, regulatory corpora, knowledge graphs, ...). Each of these sources speaks its own protocol and returns its own shape. To prevent the complexity from leaking into cortex logic, the system defines a single typed adapter contract — the `SourceFetcher` — and a registry that routes cortex requests to the right set of fetchers. This spec fixes that contract.

2 Goals

- Define one canonical `SourceFetcher` function type that all adapters must implement.
- Define a uniform `SourceResult` envelope so consumers can treat heterogeneous sources homogeneously.
- Provide a registry that maps cortex names to an ordered list of fetchers, supports self-registration, and caches results per $(cortex, params)$ key.
- Guarantee that a single fetcher failure never cancels the other fetchers of the same cortex.

3 Non-Goals

Non-Goal

This spec does not cover: the internal implementation of any specific fetcher, the schema of the `data` payload (opaque to the registry), retry and circuit-breaker policy inside a fetcher, and cortex-level fusion of fetcher results (downstream of this contract).

4 Invariants

Invariant

A `SourceFetcher` is a pure async function from an untyped parameter bag to a *list* of `SourceResult`. It may return an empty list but must never return `undefined` or `null`.

Invariant

Every `SourceResult` carries its own `type`, `source`, `fetchedAt` ISO timestamp, and `latencyMs`. These fields are non-optional.

Invariant

`fetchSourcesForCortex` is non-throwing: any rejection from an underlying fetcher is absorbed (logged, not propagated) and only the successful fetchers contribute to the returned list.

Invariant

Results are cached per $(cortex, JSON(params))$ key for `CACHE_TTL_MS`; cache hits return the exact same `SourceResult[]` reference shape as a live fetch.

Invariant

Registration is additive: `registerSource` can be called multiple times for the same cortex and preserves insertion order; it never removes a previously registered fetcher.

5 Interface

The contract is deliberately narrow: one function type, one envelope type, two registry operations.

```
export interface SourceResult {
  type: string;          // e.g. "climate.huglin", "market.index", "
     regulation.doc"
  source: string;        // human-readable origin, e.g. "open-meteo", "
     knowledge-graph"
  url?: string;          // optional deep link to the original record
  data: unknown;         // opaque payload; shape is fetcher-specific
  fetchedAt: string;     // ISO-8601 timestamp, UTC
  latencyMs: number;     // wall-clock latency of the fetch
}

export type SourceFetcher = (
  params: Record<string, unknown>,
) => Promise<SourceResult[]>;

export function registerSource(
  cortices: string[],
  fetcher: SourceFetcher,
): void;

export function fetchSourcesForCortex(
  cortexName: string,
  params: Record<string, unknown>,
): Promise<SourceResult[]>;
```

6 Scenarios

Scenario: Nominal multi-fetcher cortex

Given a cortex `C` has three fetchers registered, all of which succeed
When `fetchSourcesForCortex("C", params)` is called
Then the returned list concatenates the results of all three fetchers, in registration order
And every result carries `fetchedAt` and `latencyMs`

Scenario: One fetcher fails

Given a cortex has three fetchers and the second one throws
When `fetchSourcesForCortex` is called
Then the returned list contains only the results of the first and third fetchers
And the call resolves successfully (never rejects)
And the failure is logged but not re-raised

Scenario: Unknown cortex

Given no fetchers have been registered for cortex `"unknown"`
When `fetchSourcesForCortex("unknown", anything)` is called
Then the result is an empty array
And no fetcher is invoked

Scenario: Cache hit

Given a cortex `C` was queried with params `P` less than `CACHE_TTL_MS` ago
When `fetchSourcesForCortex("C", P)` is called again with identical params
Then the cached list is returned
And no underlying fetcher is invoked

Scenario: Cache miss on distinct params

Given the cache holds an entry for `("C", {a:1})`
When `fetchSourcesForCortex("C", {a:2})` is called
Then the cached entry is not used
And all registered fetchers for `C` are invoked

7 Acceptance Criteria

Acceptance Criterion AC-1

`fetchSourcesForCortex` for a cortex with zero registered fetchers resolves to `[]` without invoking any side effect.

Acceptance Criterion AC-2

A `SourceFetcher` that rejects with any error does not cause `fetchSourcesForCortex` to reject; the peer fetchers' results are still returned.

Acceptance Criterion AC-3

Two consecutive calls with identical `(cortex, params)` within `CACHE_TTL_MS` invoke the underlying fetchers exactly once in total.

Acceptance Criterion AC-4

Two calls with the same cortex but distinct `params` (deep-inequal by JSON) each invoke the underlying fetchers.

Acceptance Criterion AC-5

Calling `registerSource(["A","B"], f)` makes `f` contribute to both `A` and `B` results.

Acceptance Criterion AC-6

Every `SourceResult` returned by the registry has `type`, `source`, `fetchedAt` (valid ISO-8601), and `latencyMs` ≥ 0 present.

Acceptance Criterion AC-7

The function type `SourceFetcher` has exactly one arity (a single `params` argument) and always returns `Promise<SourceResult[]>` — a type-level test rejects any narrower or wider shape.

8 Traceability

AC	Test file	Test name
AC-1	`packages/thalamus/tests/source-registry.unknown.spec.ts`	`returns empty for unregister`
AC-2	`packages/thalamus/tests/source-registry.failure.spec.ts`	`absorbs single fetcher rejec`
AC-3	`packages/thalamus/tests/source-registry.cache.spec.ts`	`caches identical queries`
AC-4	`packages/thalamus/tests/source-registry.cache.spec.ts`	`bypasses cache for distinct`
AC-5	`packages/thalamus/tests/source-registry.register.spec.ts`	`registers across multiple co`
AC-6	`packages/thalamus/tests/source-registry.envelope.spec.ts`	`enforces SourceResult envelo`
AC-7	`packages/thalamus/tests/source-fetcher.types.spec-d.ts`	`SourceFetcher type is exact`

9 Open Questions

- Should the registry support cache invalidation per-cortex or per-key (currently TTL only)?
- Should `params` be typed per cortex (generic `SourceFetcher<P>`) rather than `Record<string, unknown>`?
- Should failing fetchers be surfaced to the caller as a diagnostic channel, or remain logs-only?